Evanesca: DeFi Evidence Reconstruction and Behavior Matching

Technical Report

Jeongmin Kim

This technical report documents the public Evanesca research prototype: a TypeScript framework that reconstructs DeFi transactions as Semantic Financial Graphs (SFGs), evaluates auditable behavior-matching constraints over typed financial actions, and exports replayable evidence traces for analyst review. The report is organized around the released artifact rather than a conference submission: it specifies the data model, implementation layers, DSL execution model, public JSON interface, reproduction path, and the measurement snapshot currently backed by the artifact. Evanesca is not a production detector or an attribution system. It is a forensic and measurement framework for turning heterogeneous on-chain logs into comparable financial-action traces. The current prototype uses nine constraints in 107 lines of DSL and was evaluated on 14,187,803 Ethereum transactions (2020–2024) plus 40 confirmed incidents across five chains. The artifact-backed snapshot reconstructs all confirmed incidents, flags 1,418 operational instances, separates 956 routine arbitrage baselines from 462 non-baseline instances, and records 219 source-code-validated implementation defects across derivative-pricing and reward-distribution families. The main public output is a PDF report, source code, reproducibility notes, example analysis JSON, and a schema intended for downstream evidence explorers.

Report scope. This technical report documents the Evanesca research prototype, its public artifact boundary, and the current measurement results. It is written for readers who want to inspect the method and reproduce representative evidence, not as a production detector specification. Scale-out numbers are reported with their dataset scope; implementation notes describe the open-source release state.

1 Overview and Scope

Evanesca is an open-source research prototype for reconstructing DeFi transactions as typed financial-action graphs and matching reusable behavior constraints over those graphs. The project exists to support forensic reconstruction, reproducible case studies, longitudinal measurement, and future analyst-facing evidence exploration. This document is therefore written as a technical report for the public artifact: it describes what the code does, how the DSL is evaluated, what data is emitted, which measurements are currently supported, and where the public release boundary sits.

The core design choice is to separate protocol-specific parsing from protocol-independent behavior matching. Protocol adapters translate heterogeneous logs and traces into uniform actions such as `swap`, `deposit`, `withdraw`, `borrow`, `repay`, `bridge`, and `reward`. The graph layer preserves ordering, counterparties, token movements, and USD-normalized values. The DSL layer then matches behavior over the graph: for example, a flash-loan pattern is represented as an atomic borrow/repay or explicit `FlashLoan` edge plus downstream manipulation/profit signals, rather than as a transaction-hash allowlist. Each match is a *flagged instance*, meaning an inspectable measurement signal. It is not a verdict of malicious intent.

Figure 1 is the motivating scenario used throughout this report. The transaction is not informative if it is viewed only as a transaction hash or as a bag of token transfers. It becomes explainable when the execution is reconstructed as financial actions: a temporary capital source, collateral movement, a price-moving swap, a dependent lending action, and repayment/profit realization. This example motivates the architecture in Figure 2: the system must recover semantic actions first, then preserve their order and value flow before any reusable behavior constraint can be evaluated.

This report documents the following public-facing artifact surfaces:

- **Source implementation.** The public TypeScript codebase exposes the transaction driver, semantic graph builders, DSL parser/compiler, constraint solver, and experimental JSON export CLI.
- **Evidence data model.** The public JSON schema describes SFG nodes, SFG edges, constraint findings, PnL summaries, evidence timelines, and diagnostics for downstream tools.

- **Behavior-matching DSL.** Constraints are stored as auditable DSL files and evaluated over SFG edge records; the current default attack-detection mode loads the attack-detection constraint set.
- **Measurement snapshot.** The report records the current artifact-backed numbers: 40 confirmed incidents, 14,187,803 Ethereum transactions, 1,418 flagged operational instances, and 219 source-code-validated implementation defects.
- **Reproducibility path.** Example JSON outputs, replay commands, public artifact validation, and known limitations are documented alongside the code.

The public release intentionally excludes paper-era working files, private credentials, local caches, and the LaTeX source used to build this PDF. The public repository publishes the compiled report PDF and the curated code and documentation subset needed to inspect the artifact.

2 Background and Use Model

Ledger model. Public blockchains maintain an append-only ledger of transactions readable by anyone. Smart contracts execute deterministically, making all financial activity publicly observable and reproducible. Evanesca operates on this observable record. It does not require private mempool access, builder logs, or off-chain attribution data. The system measures addresses, contracts, actions, and transaction sequences.

DeFi protocol primitives. Decentralized Finance (DeFi) protocols, autonomous financial services implemented as smart contracts, compose a small set of financial actions: Automated Market Makers (AMMs) price token *swaps* via pool invariants (e.g., $x \cdot y = k$ where x, y are token reserves); lending protocols let users *deposit* collateral, *borrow*, *repay*, and *withdraw*; *flash loans* provide uncollateralized capital that must be repaid within the same transaction; and *yield-bearing tokens* (e.g., Compound's cTokens, Aave's aTokens) are receipt tokens representing deposit positions whose exchange rate accrues over time. Price *oracles* supply external market data to contracts. Manipulating an oracle corrupts downstream pricing decisions. Every computation costs *gas*, priced in the native currency, and users pay gas fees proportional to transaction complexity. These primitives are the building blocks of both legitimate DeFi activity and the patterns we measure.

Value-Extraction Patterns. We define a *value-extraction pattern* as a repeatable sequence of *financial actions* executed to realize profit with minimal exposure. This definition is intentionally broad: routine arbitrage also fits and is retained as an explicit baseline in our evaluation. The distinction between baseline and non-baseline patterns is made during analysis: baseline patterns profit from existing price discrepancies across markets, whereas

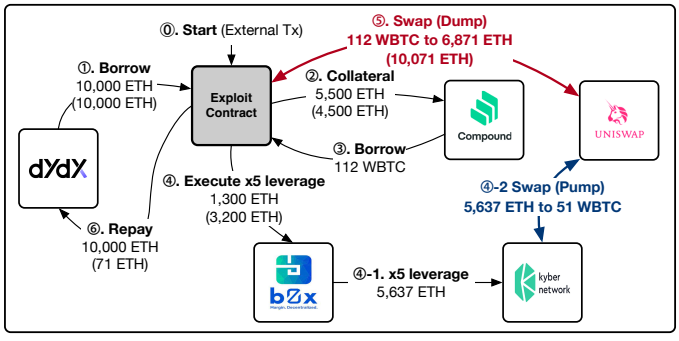

Fig. 1. Motivating scenario: representative bZx evidence trace. A flash-loan-funded manipulation step is connected to cross-protocol profit extraction.

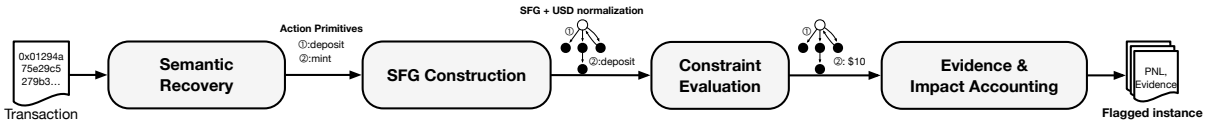

Fig. 2. System architecture. Evanesca performs semantic recovery, constructs USD-normalized SFGs, evaluates DSL behavior constraints, and exports replayable evidence traces.

non-baseline patterns require the actor to alter protocol state (e.g., inflating a token price in a low-liquidity pool) to create the profit opportunity.

Non-goals. Evanesca is not a real-time production detector, does not classify adversarial intent, does not estimate total DeFi losses, and does not provide legal, financial, or security advice. The public artifact is designed for retrospective inspection and reproducible measurement.

3 System Architecture and Evidence Model

Evanesca turns a raw transaction receipt into an analysis artifact through the four implementation stages shown in Figure 2: semantic recovery, SFG construction, DSL behavior matching, and evidence export. The figure is the system architecture used by the public artifact, not only an illustrative processing pipeline. Each stage has a narrow ownership boundary and a typed output contract. Adapters understand protocol-specific event schemas; the graph builder owns ordering, state normalization, and USD conversion; the DSL solver evaluates behavior predicates over normalized edge records; and the export layer emits evidence that can be inspected without rerunning the entire pipeline. This section specifies those data contracts.

Architecture contracts. The first contract is a sequence of recovered financial actions. Each action records a standard action type, protocol/service class, counterparties, input and output tokens, raw token amounts, and trace-order metadata. The second contract is the Semantic Financial Graph (SFG): vertices represent entities and protocol roles, while edges represent typed actions with USD-normalized values. The third contract is the constraint-evaluation result: a DSL rule name, the edge or edge sequence that matched, derived variables such as value ratio or loan size, and the predicate outcome. The final public contract is the analysis JSON consumed by examples and future frontends; it contains `summary`, `graph`, `constraints`, `pnl`, `evidence`, optional raw fields, and diagnostics. This staged design keeps protocol adapters replaceable while keeping behavior definitions reviewable and rerunnable.

Transaction ingestion and semantic recovery. The input to semantic recovery is public on-chain data: a transaction hash or batch item, its receipt, decoded logs, internal calls where available, ABI metadata, token metadata, and chain configuration. Recovery proceeds in three steps. First, protocol-specific adapters decode the raw events and call structure into candidate protocol operations. Second, the adapter groups low-level records that belong to one economic step: for example, a DEX swap may appear as several token transfers and pair events, but is emitted as one `swap` action with input/output tokens and an effective price. Third, the recovered action is emitted as an SFG-ready edge record. The public artifact covers 40+ protocol families spanning DEX, lending, flash-loan, and bridge services (Section 5). Common action primitives include `swap`, `deposit`, `withdraw`, `borrow`, `repay`, `bridge`, `reward`, and explicit `FlashLoan` when adapters expose it. Flash loans that do not expose a uniform event are represented as within-transaction `borrow+repay` pairs annotated with flash-loan metadata. That representation lets the DSL match behavior across dYdX, Aave, DODO, and forked implementations without embedding protocol-specific attack templates in the rule set.

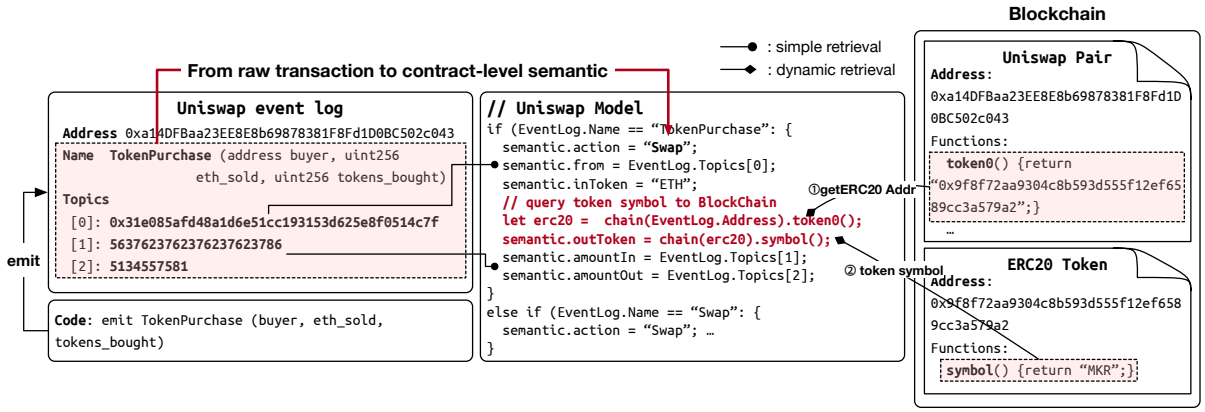

Fig. 3. Semantic recovery model. Protocol adapters map heterogeneous event logs and dynamic on-chain metadata into uniform financial-action records consumed by SFG construction.

Figure 3 shows the adapter-level contract for semantic recovery. The adapter does not decide whether an event is suspicious. It assigns financial meaning to raw data: event names and topics identify the action, static ABI knowledge gives the contract interface, and dynamic lookups such as token address and symbol retrieval complete the action record. This is the stage that absorbs protocol-specific variation. Once the output is a typed action edge, later DSL constraints can operate over `edge.Action`, token amounts, and USD values without knowing whether the source was Uniswap, Curve, a fork, or a protocol-specific wrapper.

SFG construction. The graph builder consumes recovered actions and constructs a directed graph $G = (V, E)$ for each analyzed transaction or transaction window. Vertices represent externally owned accounts, protocol contracts, pools, vaults, lending markets, and other entities that participate in value transfer. Each vertex is annotated with its address, protocol role when known, and balance state relevant to the current analysis. Edges represent typed financial actions and record input/output assets, amounts, service class, concrete protocol, counterparty roles, and the trace position that establishes ordering. Token amounts are normalized to USD using token decimals and a time-of-execution price lookup, currently through the CoinGecko price API [3]. The graph builder preserves transaction order and intra-transaction log order so that flash-liquidity cycles, manipulation-before-borrow sequences, and reward-trigger timing remain visible to later stages. This representation abstracts away protocol-specific event schemas while preserving the ordering and cross-protocol connectivity of financial actions. As a result, the same multi-step behavior remains visible even when implemented by different contracts.

More formally, an SFG can be viewed as $G = (V, E, \phi, \tau)$, where ϕ maps each edge to its recovered action semantics (`swap`, `deposit`, `withdraw`, `borrow`, or `repay`) and τ gives the strict temporal order induced by the transaction trace. Each edge carries the action type, service class, concrete protocol, input and output token amounts, USD-normalized values, and trace metadata needed for replay. This matters for measurement because the stable unit of comparison is often the financial action rather than the low-level event: a swap may appear as different event names across Uniswap, Curve, Balancer, or a fork, but it exposes the same value-preservation invariant once lifted into an SFG edge.

Edge record. The DSL sees a normalized edge record rather than raw logs. Each record groups identity fields (`Type`, `Action`, `Service`), value fields (`AmountIn`, `AmountOut`, `totalInUSD`, `totalOutUSD`), flash-loan and PnL fields (`loan_amount`, `loan_amount_usd`, `profit_usd`, `is_flash_loan`), and anomaly annotations.

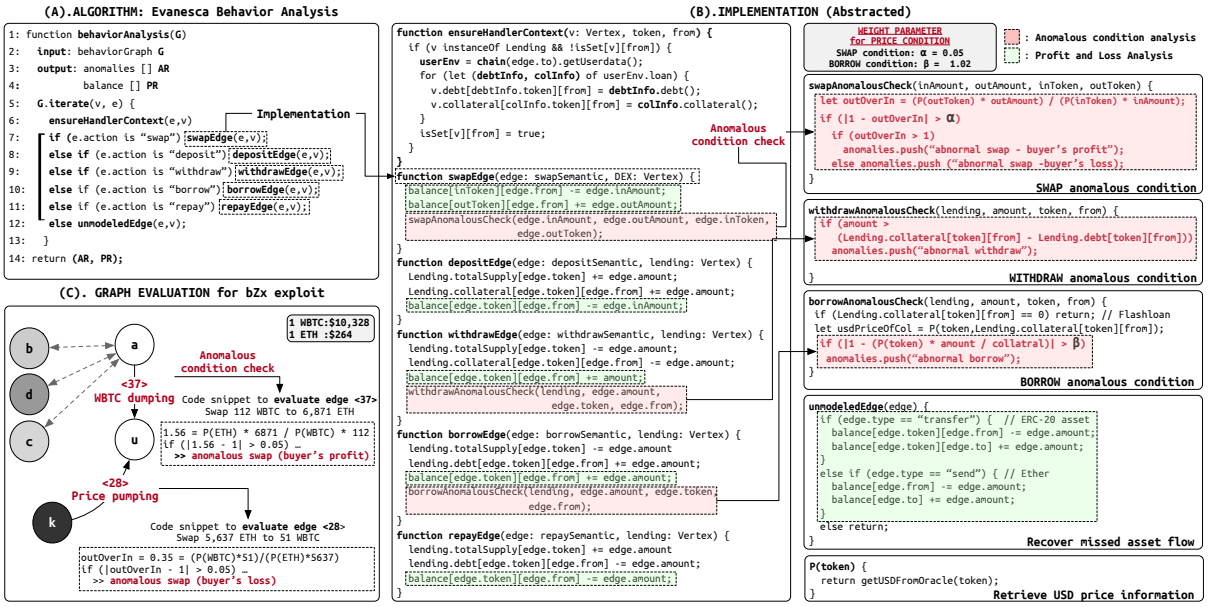

Fig. 4. Anomalous-behavior checking path. Transaction logs are lifted into a typed action graph, materialized as the SFG used by this report, and then traversed by the constraint evaluator to produce replayable evidence.

Figure 4 expands the two central modules in Figure 2: *SFG Construction* and *Constraint Evaluation*. Event indices become trace-order metadata, protocol labels become service classes, and each reconstructed action becomes an SFG edge. The same figure then shows the evaluator iterating over those typed edges and checking service-class constraints over normalized fields.

DSL behavior matching. The constraint solver consumes SFG edge records rather than raw logs. Each DSL rule has a `when` filter, derived `condition` variables, and a `violation` predicate. At load time, rules are parsed, cached, and compiled into executable functions; at run time, the solver iterates over SFG edges, skips rules whose `when` clause does not apply, computes derived variables, and records the predicate result. This makes the analysis layer explicit text rather than hidden code: a flash-loan behavior rule can match an explicit `FlashLoan` edge, a lending `Borrow` edge, or an edge annotated as flash-loan-funded by the integration layer. The rule still describes a behavior family, not a transaction-specific exploit signature. The execution path in Figure 4 is the operational model used by the public artifact: the evaluator iterates over typed action edges, dispatches service-class logic, and records matched predicates with the edge-level evidence used for replay.

We express value-extraction behavior as reusable semantic rules over SFG edges. For each edge e whose input has nonzero USD value, we compute:

$$\rho(e) = \frac{\text{valueUSD}(\text{out}(e))}{\text{valueUSD}(\text{in}(e))} \tag{1}$$

and flag edges where $|\rho(e) - 1|$ exceeds an edge-type-specific threshold. For DEX swap edges we use $\alpha = 0.05$, a conservative bound above the 1.16% average unexpected slippage on Uniswap reported by Zhou et al. [13]. For lending and oracle-mediated edges involving stablecoin derivatives we use a tighter $\beta = 0.02$, reflecting typical stablecoin oracle noise: an empirical study of DeFi oracles [6] shows that SAI and DAI daily USD price changes stay within $\leq$1% on 66–70% of days and within $\leq$5% on over 98% of days, so $\beta = 0.02$ sits just above this typical

noise floor. Our rule set comprises nine DSL constraints in 107 lines, spanning AMM invariants, lending and accounting checks, oracle anomalies, bridge integrity, reward-distribution behavior, and flash-loan patterns. The grammar and examples appear in Section 4. Protocol-specific logic is confined to the semantic recovery and SFG construction layers. The constraints themselves are protocol-independent and can be rerun over a new protocol once its adapter emits compatible edge records.

Evidence export. Given the flagged edges and the SFG, the export layer produces replayable evidence traces with per-vertex profit and loss (PnL). An evidence trace $H \subseteq G$ is the subgraph induced by the matched edge, its value-flow neighbors, and the temporal context needed to explain the match. For single-edge anomalies, this may be a compact local subgraph; for flash-loan or cross-protocol manipulation, it includes the borrow, manipulation, profit-realization, and repayment steps when those are present in the recovered graph. For each vertex v in H, we compute:

$$\begin{aligned} \mathrm{PnL}(v) = &\sum_{e \in \mathrm{in}_H(v)} \mathrm{valueUSD}(e) \\ &- \sum_{e \in \mathrm{out}_H(v)} \mathrm{valueUSD}(e) \end{aligned} \tag{2}$$

This highlights beneficiaries (positive PnL) and likely victims (negative PnL); we attribute impact across protocols, separating the protocol where the violation occurs from the entity that bears the loss. The per-step computation enables independent replay. The emitted artifact includes the matched constraint, the minimal action subgraph, the relevant $\rho(e)$ and PnL values, the derived DSL variables, and the original transaction identifier. This evidence format is the bridge to downstream tools: analysts and future frontends can render the SFG, inspect why a predicate fired, and replay the transaction from public chain data rather than relying on a transaction-level alert without intermediate computations.

Walkthrough example. The bZx incident [8, 9] illustrates the pipeline end to end. The attacker borrows via flash loan, manipulates a low-liquidity pool, and extracts profit from a lending protocol that trusts the manipulated price (Figure 1). The price-inflating swap yields $\rho = 1.56$ (56% excess output relative to input) and the victim's counterparty swap yields $\rho = 0.35$ (65% value loss), both triggering the abnormal-swap constraint. The evidence trace links these flagged edges to the attacker's profit path.

Confirmed-incident replay check. We curated a confirmed-incident set of 40 publicly documented incidents with available transaction traces across our five target chains; the set spans price manipulation attacks for DeFiRanger comparison plus reentrancy, governance, and bridge exploits for scope breadth. We define successful reconstruction as meeting three criteria: (i) the evidence trace identifies the correct manipulation step, (ii) it traces a profit path to the beneficiary, and (iii) at least one general-purpose constraint fires on the value-extracting edge. Across all 40 confirmed incidents, our pipeline meets these criteria. Table 1 summarizes incident diversity and constraint-family coverage; the detailed per-incident inventory appears later in Table 3. Price-deviation and AMM constraints each fire on 55.0% of incidents, followed by reentrancy and collateral/accounting at 30.0% each.

Replay check. The nine protocol-independent rules reconstruct 40/40 confirmed incidents across five chains without per-incident logic. This check validates that the public artifact can recover known evidence traces before being used for scale-out measurement.

4 DSL Behavior Matching

Purpose and execution model. Evanesca uses the DSL as a behavior-matching layer over normalized SFG edge records. Rules are stored as text files under `src/DSL/constraints/`; the default attack-detection mode loads `attack_detection_constraints.dsl` through the dynamic constraint loader. Each rule is parsed into an AST,

Table 1. Confirmed incidents: distribution by attack type and coverage by constraint family.

Attack type	Count	Constraint family	Incidents*
Flash loans (incl. flash-loan+price)	13	Price-deviation constraints	22 (55.0%)
Price manipulation	11	AMM constraints	22 (55.0%)
Reentrancy	7	Collateral/accounting constraints	12 (30.0%)
Oracle manipulation	4	Exchange-rate constraints	9 (22.5%)
Bridge exploits	2	Oracle integrity constraints	7 (17.5%)
Other (logic, governance, donation)	3	Bridge / flash-loan constraints	1 (2.5%) each
Total	**40**		

*Percentages sum to >100% because one incident may trigger multiple constraint families.

cached, and compiled into JavaScript functions for `when`, `condition`, and `violation` evaluation. This keeps the matching logic auditable while avoiding repeated parsing during batch runs.

Constraint format. Each constraint declares (i) when it applies, (ii) derived variables, and (iii) a violation predicate:

```
constraint NAME {
  when: <boolean expression over edge fields>
  condition: { x: <expr>, y: <expr>, ... }
  violation: <boolean expression>
  message: "human-readable summary"
}
```

`when` filters candidate edges before any expensive predicate work; `condition` binds intermediate expressions by name; `violation` decides whether the behavior matched; and `message` is attached to the finding for triage. Expressions support arithmetic and boolean operators (`+`, `-`, `*`, `/`, `<`, `<=`, `==`, `&&`, `||`) and member access such as `edge.Type` and `edge.totalInUSD`. In `condition` bindings, `||` serves as a fallback operator: for example, `x || 0` yields `x` if defined, else `0`. In `violation` expressions, `||` is logical OR.

Evaluation context. The DSL is evaluated over the per-edge records emitted by semantic recovery and USD normalization. In the examples below, `edge` refers to the current SFG edge and exposes fields such as `Type`, `Action`, `Service`, `totalInUSD`, `totalOutUSD`, `loan_amount_usd`, `profit_usd`, and `is_flash_loan`. Multi-step behavior is represented by annotations and temporal context computed by the graph builder, flash-loan integration layer, and pattern analyzers. The DSL therefore matches behavior over semantic evidence, not raw ERC-20 transfer strings. The examples below are representative excerpts from the public constraint model. They are intentionally written as behavior predicates rather than exploit signatures: the same rule can match multiple incidents if their recovered SFG edges expose the same economic structure. In release runs, adapters may attach additional fields to the edge record, but the DSL boundary remains the same input contract for downstream auditing.

Abnormal swap behavior. The first example computes a value ratio `price_ratio` (equivalently, $\rho(e)$ = valueUSD(out)/valueUSD(in)) and matches abnormal exchange-rate behavior on DEX swap edges.

```
constraint PRICE_MANIPULATION {
  when: edge.Type == "DEX" && edge.Action == "Swap"
  condition: {
    price_ratio: edge.totalOutUSD / edge.totalInUSD,
    price_impact: edge.price_impact_percent || 0,
    is_atomic: edge.is_atomic_transaction || false
  }
  violation:
    edge.totalInUSD > 0 &&
    (price_ratio > 1.05 ||
```

```
    (price_impact > 10 && is_atomic))
  message: "abnormal exchange rate"
}
```

Production constraints include additional guards where adapter fields are available, but the structure remains the same: filter candidate actions, derive behavior features, and emit a match when the predicate fires.

Flash-loan behavior. Flash loans are a good example of behavior matching. The rule does not name a specific incident. It matches explicit `FlashLoan` actions, lending `Borrow` edges, or edges annotated as flash-loan-funded by the integration layer, then checks for large loan size, profit, manipulation flags, or excessive borrowing.

```
constraint FLASH_LOAN_ATTACK {
  when: edge.Action == "FlashLoan" ||
        (edge.Type == "Lending" &&
         edge.Action == "Borrow") ||
        edge.is_flash_loan == true
  condition: {
    loan_size: edge.loan_amount_usd,
    profit: edge.profit_usd,
    manipulated_protocol:
      edge.price_manipulated ||
      edge.oracle_manipulated,
    excessive_ratio:
      (edge.borrow_amount_usd /
       edge.borrow_available) * 100
  }
  violation:
    (loan_size > 100000 && profit > 10000) ||
    (edge.is_flash_loan &&
     manipulated_protocol) ||
    (edge.borrow_amount_usd > 0 &&
     edge.borrow_available > 0 &&
     excessive_ratio > 1000)
  message: "Flash loan attack pattern detected"
}
```

In other words, the DSL can express the behavior family "borrow capital atomically, manipulate a protocol state variable, and exit with profit" once the SFG layer has annotated the relevant action edges. The same rule can fire on different providers and protocols because the provider-specific parsing is isolated in semantic recovery.

Accounting and bridge examples. The shipped rule set also covers lending collateralization, exchange-rate manipulation, AMM invariant breaks, oracle manipulation, concentrated-liquidity manipulation, bridge integrity violations, and empty market manipulation. Rules are intentionally reviewable text rather than opaque code paths; analysts can inspect the predicate that fired and then replay the corresponding evidence subgraph. For readability, examples in this report may wrap long lines or shorten message strings, while preserving the relevant predicate structure.

5 Implementation and Public Artifacts

Evanesca is implemented as a modular TypeScript framework. The public codebase is organized around the same stages described in Section 3: event decoding, semantic graph construction, DSL behavior matching, and evidence export. The implementation goal is to make transaction-level evidence reproducible from public chain data while keeping batch execution practical at multi-year scale.

Primary code paths. The main transaction path is organized around explicit implementation boundaries:

- `src/Driver.ts` retrieves event logs and coordinates the analysis context.
- `src/ABIDecoder/` decodes raw logs into protocol-level events.
- `FormalSemanticFinancialGraphBuilder` lifts decoded events into SFG nodes and edges.
- `DSLConstraintSolver` evaluates compiled DSL constraints and records solver output.
- `src/DSL/` contains the lexer, parser, compiler, cache, and DSL runtime.

- `src/DSL/constraints/` contains the public constraint files.
- `src/ConstraintSolver/` contains dynamic loading and result assembly.
- `src/cli/analyze.ts` exports one transaction through the public JSON interface.

Repository map. The public repository is intended to be navigable without reading the paper-era working tree. The main directories are:
- `src/ABIDecoder/`: ABI registry and event-log decoding.
- `src/SemanticFinancialGraph/`: SFG builders, protocol adapters, edge adders, and graph utilities.
- `src/DSL/`: lexer, parser, compiler, interpreter, mathematical extensions, and constraint files.
- `src/ConstraintSolver/`: dynamic constraint loading, per-edge evaluation, flash-loan integration, and solver result assembly.
- `src/PatternDetection/`: auxiliary multi-step analyzers used outside strict DSL-only evaluation.
- `src/cli/`: public command-line wrappers for benchmark and JSON export paths.
- `docs/public-api/`, `docs/reproducibility/`, and `docs/example-datasets/`: stable public documentation surfaces for downstream users.

Semantic adapters. Each supported protocol family has a dedicated adapter that maps its event schema to SFG action edges. We currently ship adapters for 40+ protocol families spanning DEX (Uniswap V1/V2/V3, SushiSwap, PancakeSwap, Curve, Balancer, Bancor, KyberSwap, Synthetix, Platypus, WooFi, and others), lending (Compound, Aave, Cream Finance, Venus, Inverse, Euler, bZx, Radiant, Hundred, and others), flash-loan (dYdX, AaveV3, DODO), and bridge protocols (Qubit, Allbridge), covering constant-product AMMs, stableswap, concentrated liquidity, lending, flash-loan, and cross-chain primitives. Adding a new protocol requires implementing one adapter module; existing constraints and evidence extraction remain unchanged. Adapters emit typed action edges rather than detector verdicts. This separation is intentional: protocol-specific logic stays in semantic recovery, while constraint definitions remain service-class level and can be rerun over different datasets without rewriting exploit templates.

Constraint loading and execution modes. Constraints are not hardcoded in the driver. The dynamic constraint loader selects DSL files according to execution mode. Default attack-detection mode loads the public attack-detection constraint file. Protocol-verification mode loads protocol invariants, and `DSL_FILE` can point the solver to a custom DSL file. The solver supports a strict DSL-only mode for runs that must exclude auxiliary custom detectors. This distinction is important for reproducible evaluation: the report numbers refer to the DSL-backed constraint path, while auxiliary pattern analyzers are implementation helpers for broader tooling.
- Default: `attack_detection_constraints.dsl`
- Protocol verification: `protocol_invariants.dsl`
- Custom run: `DSL_FILE=/path/to/rules.dsl`

Execution configuration. The most important public execution knobs are environment variables rather than code edits.
- `DSL_FILE`: selects a specific DSL file for a run.
- `EVANESCA_DSL_ONLY=true`: disables auxiliary custom detectors and multi-step shortcuts so the output reflects DSL formula evaluation.
- `PROTOCOL_VERIFICATION_MODE=true`: switches to protocol-invariant checking.
- `MICA_REGULATION_MODE=true`: selects the MiCA regulatory constraint set, which is outside this report's DeFi value-extraction snapshot.
- `EVANESCA_QUIET=true`: suppresses most non-error logs.
- `DEBUG_EVANESCA`: enables component-level debugging.

Historical replay additionally needs RPC/API credentials such as `ETHERSCAN_API_KEY`, `BSCSCAN_API_KEY`, or provider-specific RPC keys configured through `.env`.

Ingestion and chain configuration. The collector uses a unified EVM ingestion interface with chain-specific configuration for block timing, RPC/log retrieval, and token metadata. The scale-out study uses Ethereum for the 14.2M-transaction measurement and retrieves confirmed-incident traces from BSC, Arbitrum, Avalanche, and Optimism for incident-reconstruction coverage. The same ingestion abstraction is designed to support additional EVM-compatible networks when their traces and token metadata are available.

Public JSON interface. The public CLI exports a schema-oriented JSON artifact:

```
npm run analyze -- --tx <hash> --chain ethereum \
  --out out.json --pretty
```

The current schema, documented in `docs/public-api/analysis-result.schema.md`, includes summary metadata, SFG nodes and edges, constraint matches, PnL attribution, evidence subgraphs, optional raw fields, and diagnostics. The intended frontend should consume this JSON contract rather than importing internal TypeScript modules directly.

Common commands. The public workflow is intentionally small:

- `PUPPETEER_SKIP_DOWNLOAD=true npm install`: install dependencies without downloading a browser that the CLI path does not need.
- `npm run validate:public-artifacts`: verify that required public artifacts and report files are present.
- `npm run analyze`: export one transaction as public-schema JSON using the options shown above.
- `npm run attack`: run the confirmed-incident benchmark; this can be slow and usually requires archive-capable RPC access.
- `npm run large-scale` and `npm run large-scale:ingest`: run research-scale measurement paths when the required data and cache environment are available.

Scalability and batch measurement. For each transaction, Evanesca evaluates all DSL constraints on every SFG edge. The current rule set contains nine constraints in 107 lines of DSL. Each constraint includes a `when` clause that acts as a lightweight type filter; for example, a DEX invariant fires only on swap edges. Irrelevant constraints are then skipped without full evaluation. Performance comes from three optimizations: (i) DSL constraints are compiled to JavaScript functions at load time, avoiding repeated parsing; (ii) parsed constraint ASTs are cached with SHA-256 hashing; and (iii) token-to-USD conversions are batched before constraint evaluation to avoid redundant price lookups. The evaluator is staged: inexpensive type filters and single-edge invariants run first, and only surviving candidates trigger more expensive temporal correlation and evidence-subgraph extraction. This keeps the number of analyst-facing artifacts manageable while preserving the intermediate computations needed for reproducibility.

USD normalization. Resolved CoinGecko prices [3] are cached per (token, timestamp) pair with LRU eviction and bounded TTL to reduce API calls during batch processing. CoinGecko exposes historical prices indexed by timestamp, so the same transaction can be re-priced consistently across runs as long as the API remains accessible. Alternative historical-price sources (e.g., CoinMarketCap's historical API [4], Chainlink on-chain price feeds [2]) could replace CoinGecko without changes to the rest of the pipeline.

Deployment and throughput. Our distributed deployment uses 3 machines (Intel Xeon E5-2640 v4 @ 2.40 GHz, 128 GiB RAM each) running 30 Docker workers, 10 per machine. Mean per-transaction processing time is 309 ms ($p95 < 1.2$ s; max 9.9 s), keeping multi-year measurement practical while preserving reproducible evidence for flagged instances.

Public release boundary. The public repository contains source code, documentation, example datasets, reproducibility notes, public artifact manifests, the analysis-result schema, and the compiled technical report PDF. It does not publish private RPC credentials, local caches, paper drafting branches, or the LaTeX sources used to generate this PDF. Public validation currently runs the artifact validator, CLI help path, and release smoke tests; full historical replay requires archive-capable RPC credentials.

6 Measurement Snapshot and Case Studies

This section records the current artifact-backed measurement snapshot. It is included to document what the released prototype has already reproduced, not to present a new benchmark leaderboard. The scale-out numbers are the current validated artifact-backed numbers; no additional large-scale claims are introduced in this technical report. The snapshot covers 14,187,803 Ethereum transactions (2020–2024) and 40 confirmed incidents spanning five chains.

6.1 Dataset Scope

Scope and collection. For January 2020 through December 2024, we collect all Ethereum transactions matching our 40+ adapter set (Section 5), yielding 14,187,803 transactions. For the remaining four chains (BSC, Arbitrum, Avalanche, and Optimism), we retrieve only the transactions associated with our 40 confirmed incidents; these chains provide incident-reconstruction coverage but not the Ethereum scale-out measurement. Token values are normalized to USD at each transaction's block timestamp (Section 5). Instance counts should be interpreted as results over this Ethereum dataset, not as multi-chain population prevalence estimates.

6.2 Operational Pattern Snapshot

We flag 1,418 instances across 14,187,803 transactions using the nine DSL constraints. Table 2 summarizes the distribution across pattern categories after separating routine arbitrage from non-baseline candidates.

Baseline separation. Of the 1,418 flagged instances, 956 are routine DEX arbitrage: they exceed $\alpha = 0.05$ but evidence traces show no flash-loan or oracle/lending constraint hit, so we retain them as benign baseline (Figure 5, Panel A). The remaining 462 non-baseline instances combine the trigger with at least one such hit, examined below. We manually validated this partition by cross-checking each instance's Etherscan event log against its SFG, consulting smart-contract source when action-level evidence was insufficient (Section 6.5).

Table 2. Breakdown of flagged instances at scale, including baseline.

Category	Count	Example signal
Price-discrepancy arbitrage (baseline)	956	$\lvert\rho(e) - 1\rvert$ large
Reward-distribution manipulation	207	transient in/out around reward trigger
Price manipulation	166	clustered abnormal swaps
Yield-bearing token deviation	12	persistent share-price gap
Other non-baseline patterns	77	long tail (mixed signals)
Total	**1,418**	

Non-baseline categories. We group the 462 non-baseline instances by extraction mechanism, yielding three named families plus a long tail. *Price manipulation* (166 instances) uses flash-loan capital to inflate a token's price on a liquidity pool, then sells previously held tokens at the inflated price on the same pool (Figure 5, Panel B). *Yield-bearing token deviation* (12 instances) reflects gaps between on-chain exchange rates and fair redemption values for derivative tokens. The profit path runs through cross-protocol deposit/withdraw cycles rather than a single corrective swap (Figure 5, Panel C). Section 6.3 gives step-by-step walkthroughs for Panels B and C. *Reward-distribution manipulation* (207 instances) uses flash-loan-funded momentary deposits to capture reward

shares without sustained holding (Figure 6). The remaining 77 instances form a long tail that does not cluster into the three named families above.

Snapshot. At 14.2M-transaction scale, the same nine constraints flag 1,418 instances forming a measured pattern landscape: 956 routine arbitrage and 462 non-baseline transactions spanning three distinct families plus a long tail. This resolution shows that DeFi value extraction at scale is structurally diverse, not a single attack template scaled up.

6.3 Representative Pattern Notes

Figure 5 gives representative evidence traces for the baseline class and the two non-baseline arbitrage families most likely to be confused with routine arbitrage. The notes below summarize the measured action sequence and the constraint signal used for classification.

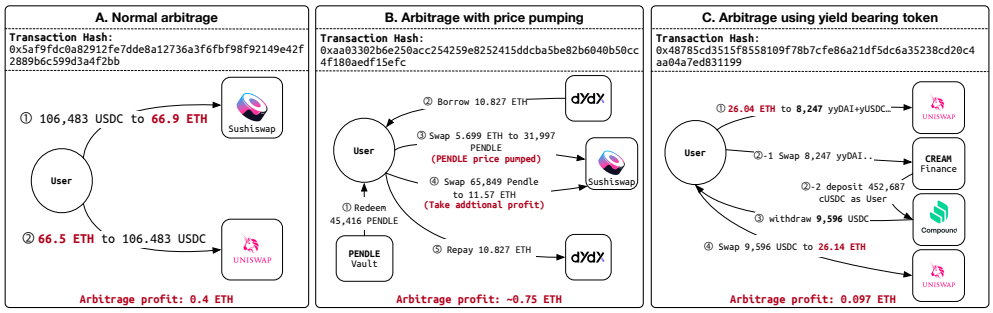

Fig. 5. Representative arbitrage patterns used for the scale-out classification: routine arbitrage baseline (A), price pumping (B), and yield-bearing token deviation (C).

Panel A: Routine Arbitrage. The baseline trace is a closed swap cycle that starts and ends in the same asset after crossing two DEX venues. It produces a positive spread and may trigger a large swap-ratio signal, but the evidence trace contains no flash-loan-funded state manipulation, no lending/oracle edge, and no persistent share-price or reward-accounting deviation. We therefore retain it as value-extractive but benign baseline activity: the transaction profits from an existing market discrepancy rather than creating the discrepancy by altering protocol state.

Panel B: Price Manipulation. The actor borrowed 10.827 ETH from dYdX, used part of the capital to buy 31,997 PENDLE on Sushiswap, and then sold a larger pre-existing PENDLE position into the inflated pool price before repaying the flash loan. The measured net gain was approximately 0.75 ETH in one transaction. The trace fires `PRICE_MANIPULATION` on the abnormal swap ratio and `FLASH_LOAN_ATTACK` on the atomic borrow–manipulate–repay cycle.

Panel C: Yield-Bearing Token Deviation. The actor bought Yearn vault-share tokens on Uniswap, deposited them into CreamY, and received Compound cUSDC at a value roughly 2.0% above the fair share price. The cUSDC was then redeemed through Compound and swapped back to ETH, producing approximately 0.097 ETH per cycle. The trace fires `EXCHANGE_RATE_MANIPULATION`: the cUSDC redeem edge unwraps a position acquired through CreamY at Compound's fair per-share rate, exposing a cross-protocol exchange-rate gap rather than a simple two-pool arbitrage.

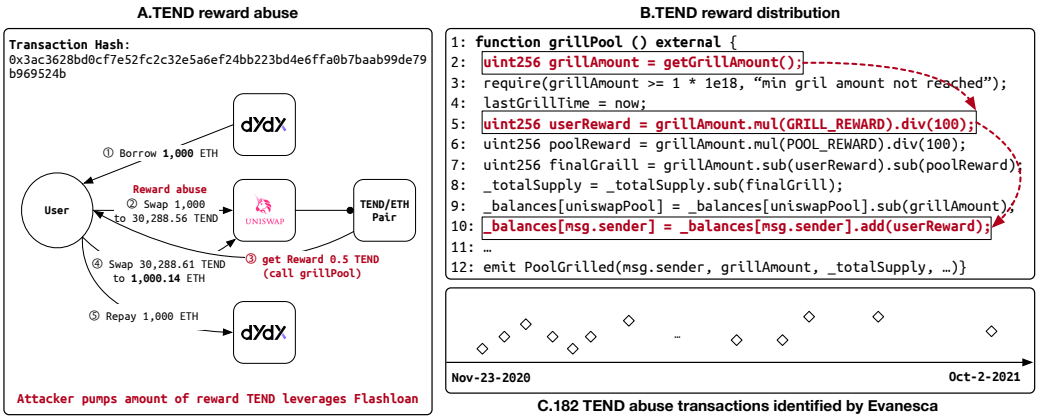

Fig. 6. Representative reward-distribution manipulation (TEND). A flash-loan-funded swap triggers reward payout and an immediate unwind.

6.4 Validated Defect Families

We inspected all 462 non-baseline instances via event-log and source-code analysis, independently of the extraction-mechanism grouping in Section 6.2. Of the 462 inspected instances, 219 have a harm-causing mechanism we identified in source code, and split into two defect families: all 12 yield-bearing cases map to derivative pricing errors and all 207 reward-manipulation cases to distribution bias. For the remaining 243 (166 price manipulation + 77 long-tail), source-code inspection revealed no implementation defect—extraction is driven by attackers exploiting otherwise correct service logic.[1] Manual inspection of all 956 baseline instances found no harmful activity (Section 6.5), validating this partition as a reliable triage primitive.

Exchange-rate computation errors. Cross-protocol arbitrageurs captured exchange-rate gaps via deposit/withdraw cycles, unnoticed by protocol operators. Twelve stablecoin-backed derivative tokens (cUSDC and yyDAI families) showed 2–2.5% mispricing per instance, caused by systematic on-chain rate errors where the on-chain rate is computed from the derivative token's own contract and misses deviations at the same token's other trading venues. Our detection catches these at $\beta = 0.02$ over lending/oracle-mediated edges (Section 3).

Reward-distribution bias. Several contracts compute payouts based on the balance at the moment of interaction rather than time-weighted holdings, enabling flash-loan-funded deposits that earn disproportionate rewards within a single transaction. Evanesca flags 207 instances, all concentrated in the TEND and WING contracts (shared deflationary logic), cumulatively yielding 15.31 ETH to attackers. Figure 6 illustrates a representative instance (TEND): a flash-loan-funded swap triggers excess reward issuance, and the attacker exits via a reverse swap. The two steps occur in different contracts, requiring cross-protocol evidence traces. Currently both TEND and WING contracts (as well as the yield-bearing wrappers in the previous subsection) are either immutable or no longer actively maintained, limiting the scope for responsible disclosure.

Snapshot. Of the 462 non-baseline instances, 219 (47%) correspond to real implementation defects in two previously uncatalogued families: derivative pricing errors and reward-distribution bias. The same nine constraints thus extend from incident reconstruction to defect discovery, without per-defect tuning.

[1]We searched DeFiHackLabs, Rekt News, SoK: DeFi Attacks [14], and audit reports for the affected tokens; neither defect family appears in prior aggregated measurement literature.

6.5 Flagged Instance Validation and Impact Accounting

Operational measurement yields flagged instances; turning these into actionable findings requires additional validation. We use the following workflow, reproducible from public chain data: (i) re-extract the evidence trace from the transaction identifier and confirm that computed metrics such as $\rho(e)$ and PnL are stable under re-execution; (ii) verify that implied profit exceeds fees/gas and that the value transfer is not a pure accounting artifact; (iii) verify that the operation remains profitable under different reasonable price assumptions, reducing dependence on a single price source.

Impact accounting. We report impact in units native to each extraction path rather than USD totals, avoiding dependence on volatile price sources. For exchange-rate/share-price errors, we express the per-extraction rate gap as a percentage. For reward-distribution manipulation, we compute the attacker's net profit after all borrowed capital is repaid (e.g., flash-loan repayment); the remaining tokens represent value extracted from the protocol's reward pool. These figures are conservative lower bounds and should be interpreted as measurement signals grounded in replayable evidence rather than as definitive accounting.

Manual audit of baseline instances. We manually inspected all 956 baseline instances by cross-checking each transaction's Etherscan event log against Evanesca's SFG (and smart-contract source when action-level evidence was insufficient). None met our criteria for harmful activity (price manipulation, protocol exploitation, or other adversarial behavior). All instead exhibited cross-protocol price discrepancies (large $|\rho(e) - 1|$ values) with rapid position unwinding consistent with routine arbitrage.

6.6 Prior-System Compatibility Check

The technical report keeps a compact compatibility check against DeFiRanger, a closely related academic framework for detecting DeFi price manipulation attacks. As it is closed-source, we adopt DeFiScope's per-incident verdicts [12] for the 12 overlapping incidents and apply DeFiRanger's published Tr1→Ba→Tr2 pattern for the remaining 28.

Coverage results. Our constraints detect all 40. DeFiRanger covers 12 of 40 (Section 6.6.1). The 23 out-of-scope misses cluster across reentrancy, governance, bridge, oracle defects, admin-key compromise, direct fund injection, and logic-bug mechanisms. The 5 within-scope misses—Pancake Bunny, CreamFinance #2, EGD Finance, Elephant Money, and Inverse Finance—are PMAs per [12] that DeFiRanger fails on, exposing CFT brittleness across flash-mint chains, share-price defects, and asymmetric (no-Tr2) extractions. Our approach detects all five because constraints evaluate each edge independently.

Within-scope comparison. We restrict the comparison to the 17 incidents within DeFiRanger's price manipulation scope: DeFiRanger covers 12 (70.6%) and our constraints detect 17 (100%). The gap reflects both coverage breadth and within-scope pattern brittleness. DeFiScope itself misses Harvest #1 (compilation), Indexed Finance (LLM-computation), and Inverse Finance (cross-transaction) among the 12 overlapping incidents [12]. Evanesca catches all three. We can extend Evanesca to a new protocol family by adding only a semantic adapter, without new detection templates (Section 6.6.1).

Architectural difference. DeFiRanger builds cash-flow trees (CFTs) from raw ERC-20 token transfers and pattern-matches Transfer–Balance-change–Transfer (Tr1→Ba→Tr2) sequences, where Tr1 acquires capital, a balance change alters internal state, and Tr2 extracts value at the altered state. Unlike DeFiRanger, our approach constructs SFGs with typed financial actions such as `swap`, `deposit`, and `borrow`, and expresses detection logic as DSL constraints over these edges (Section 4). This enables constraint categories such as reentrancy, governance, bridge, and oracle defects that fixed transfer-level templates cannot express. This explains the 23 out-of-scope misses above: their extraction mechanism is invisible at the transfer level.

6.6.1 Per-Incident Comparison Details. This section provides per-incident outcomes and a constraint-family breakdown for the DeFiRanger comparison summarized in Section 6.6. DeFiRanger is closed-source. We adopt per-incident verdicts from DeFiScope [12] for the 12 incidents in their dataset that overlap with ours, and apply DeFiRanger's published Tr1→Ba→Tr2 transfer pattern for the remaining 28, classifying each as in-scope detected, in-scope missed, or out-of-scope.

Table 3 reports the per-incident inventory. Evanesca reconstructs every incident in the set, so the table focuses on the DeFiRanger outcome. We mark an incident as *detected* when its transfer sequence matches DeFiRanger's published Tr1→Ba→Tr2 cash-flow-tree pattern; as *missed* when it is a price manipulation but the published pattern's structural requirements fail to fire, as in bZx's split capital acquisition; and as *out* when it lies outside DeFiRanger's published price-manipulation focus.

Incidents whose value extraction relies on other mechanisms – reentrancy, governance, bridge accounting, share-price oracle defects, admin-key compromise, direct fund injection into pool reserves, or pure logic bugs – are marked *out*. The 23 out-of-scope misses cluster into seven mechanism groups: reentrancy (CreamFinance #1, Origin Protocol, Akropolis, Rari Capital, Curve Finance, and dForce via read-only reentry on Curve), governance (BeanstalkFarms and Fortress Loans, the latter combined with oracle abuse), bridge accounting (Qubit Finance and Allbridge), share-price or oracle-misconfiguration defects (Yearn Finance, Radiant Capital, and Concentric), admin-key compromise (Rikkei Finance and Crosswise, where the oracle was overwritten directly), direct fund injection (Euler Finance and Hundred Finance, where donated funds inflate share price without a swap step), and pure logic bugs (KyberSwap, Platypus Finance, BlueberryFDN, Shido Global, Saddle Finance virtual-price arithmetic, and MIM Spell cauldron precision). The 5 within-scope misses (Pancake Bunny, CreamFinance #2, EGD Finance, Elephant Money, Inverse Finance) are discussed in Section 6.6; the compact inventory follows as a row-level audit log rather than a detector leaderboard.

7 Limitations and Comparison Notes

Prior-system boundary. Existing academic and industry systems often serve different purposes: symbolic oracle analysis [10], asset-flow diagrams for individual flash-loan events [1], exploit parameter optimization [9], price-manipulation-specific frameworks such as DeFort [11] and DeFiScope [12], and operational monitors such as Forta [5] and OpenZeppelin Defender [7]. Evanesca is not a replacement for those systems. It is a replayable evidence pipeline and behavior-matching framework that keeps the matched predicate, SFG subgraph, and value-flow calculation visible to the analyst.

Coverage limits. Our confirmed-incident set is curated from public post-mortem reports and may not represent all attack categories. Our semantic adapters cover major AMM, lending, flash-loan, and bridge protocols but not all DeFi primitives (e.g., options, perpetuals). Patterns in unsupported protocols would be missed. The $\alpha = 0.05$ threshold is grounded in prior findings [13], set conservatively above the 1.16% average unexpected slippage on Uniswap reported there. Because confirmed defects span multiple constraint families beyond the swap-ratio check, the defect inventory is not dominated by this single parameter. A formal α/β sensitivity surface is deferred to deployment-specific calibration.

Operational limits. The nine constraints target invariants over DeFi primitive actions and may miss purely computational vulnerabilities (e.g., access-control bypass) that do not alter observable financial outputs. The 219 confirmed defects (47% of 462) are also a conservative floor; the remaining 243 are price manipulation attacks or long-tail patterns with no identifiable protocol code defect. Historical replay depends on archive-capable RPC access, log availability, token metadata, and historical price sources. Provider differences can change diagnostics or USD attribution even when the semantic action trace is stable.

Table 3. Scope-aware incident inventory used for the confirmed-incident comparison. Evanesca reconstructs all 40 incidents; the final column records DeFiRanger status.

#	Incident	Loss	Mechanism	Chain	DeFiRanger
2020–2021					
1	CreamFinance #1	$18.8M	Collat., exch. rate	ETH	out
2	Harvest #1	$24M	Exch. rate	ETH	detected
3	bZx Hack	$350K	AMM, collat., oracle, price	ETH	detected
4	Harvest #2	$10M	Exch. rate	ETH	detected
5	Origin Protocol	$7M	Collat., exch. rate	ETH	out
6	Value DeFi	$6M	AMM, price	ETH	detected
7	Warp Finance	$7.7M	Price	ETH	detected
8	Akropolis	$2M	Collat., exch. rate	ETH	out
9	Cheese Bank	$3.3M	AMM, price	ETH	detected
10	Yearn Finance	$11M	Collat., oracle	ETH	out
11	xToken #2	$24.5M	AMM, price	ETH	detected
12	Pancake Bunny	$45M	AMM, price	BSC	missed
13	CreamFinance #2	$130M	Collat.	ETH	missed
14	Indexed Finance	$16M	AMM, price	ETH	detected
2022					
15	BeanstalkFarms	$182M	AMM, price	ETH	out
16	Qubit Finance	$80M	Bridge	BSC	out
17	Rari Capital	$80M	Collat., exch. rate	ETH	out
18	EGD Finance	$36M	AMM, price	BSC	missed
19	Inverse Finance	$15.6M	AMM, price	ETH	missed
20	Elephant Money	$11.2M	AMM, price	BSC	missed
21	Float Protocol	$1.4M	AMM, price	ETH	detected
22	Saddle Finance	$10M	Collat.	ETH	out
23	Fortress Loans	$3M	Collat., flash loan	BSC	out
24	Rikkei Finance	$1.1M	AMM, price	BSC	out
25	Crosswise	$0.9M	AMM, collat., exch., price	BSC	out
26	Wiener DOGE	$0.1M	AMM, price	BSC	detected
2023					
27	Euler Finance	$200M	Collat., exch. rate	ETH	out
28	Curve Finance	$41M	AMM, price	ETH	out
29	KyberSwap	$48M	AMM, price	ETH	out
30	Hundred Finance	$7M	Collat., exch. rate	OP	out
31	Platypus Finance	$8.5M	AMM, price	AVAX	out
32	Allbridge	$0.6M	AMM	BSC	out
33	dForce	$3.7M	Oracle	ARB	out
2024					
34	Radiant Capital	$4.5M	Oracle	ARB	out
35	WooFi Swap	$8.8M	AMM, oracle, price	ARB	detected
36	Gamma Strategies	$3.4M	AMM, oracle	ARB	detected
37	MIM Spell	$6.5M	AMM, price	ETH	out
38	BlueberryFDN	$1.4M	Price	ETH	out
39	Concentric	$1.8M	Oracle	ARB	out
40	Shido Global	$3.3M	AMM, price	BSC	out
Evanesca overall reconstruction					**40/40**
DeFiRanger overall detection					**12/40**
DeFiRanger within PMA scope					**12/17**

Release limits. The public release is intentionally conservative. It publishes source code, example datasets, reproducibility notes, public artifact validation, and this compiled PDF, but it does not claim full large-scale reproducibility without the original large-scale data environment. The current frontend contract is the JSON schema; live UI integration is a separate future artifact.

8 Release Summary

This technical report packages Evanesca as a public research artifact rather than as a conference-paper submission. The released system reconstructs Semantic Financial Graphs from on-chain activity, evaluates auditable DSL behavior constraints over typed financial actions, and exports evidence suitable for replay, inspection, and future frontend visualization. The current artifact-backed snapshot covers 14,187,803 Ethereum transactions, 40 confirmed incidents, 1,418 flagged operational instances, and 219 source-code-validated implementation defects.

Those numbers document the current prototype boundary; the intended long-term value of the release is the reusable implementation, DSL, evidence schema, and reproducibility path for DeFi value-extraction analysis.

Acknowledgments

Open-source release note. The public release includes source code, documentation, reproducible artifact manifests, and example JSON exports for bZx and Harvest replayed transactions. The compiled technical report PDF is intended to document the research prototype and measurement results; it is not a production security guarantee or an automatic attribution system.

References

[1] Yixin Cao, Chuanwei Zou, and Xianfeng Cheng. 2021. Flashot: A Snapshot of Flash Loan Attack on DeFi Ecosystem. *arXiv preprint arXiv:2102.00626* 1 (2021), 1 – 25.

[2] ChainLink. [n. d.]. Off-chain price oracle - securely connect smart contracts with off-chain data and services. https://chain.link/. Accessed: 2026-04-20.

[3] CoinGecko. 2024. CoinGecko API: Historical Cryptocurrency Prices and Market Data. https://www.coingecko.com/en/api. Accessed: 2026-04-16.

[4] CoinMarketCap. [n. d.]. CoinMarketCap API: Historical Cryptocurrency Prices and Market Data. https://coinmarketcap.com/api/documentation/. Accessed: 2026-04-20.

[5] Forta Foundation. 2022. Forta: A Decentralized Runtime Security Solution for Automated Threat Detection and Prevention. https://docs.forta.network/en/latest/2022-7-11%20Forta%20Litepaper.pdf. Forta Litepaper. Accessed: 2026-04-06.

[6] Bowen Liu, Pawel Szalachowski, and Jianying Zhou. 2021. A First Look into DeFi Oracles. In *2021 IEEE International Conference on Decentralized Applications and Infrastructures (DAPPS)*. IEEE, 39–48.

[7] OpenZeppelin. 2024. OpenZeppelin Defender: Smart Contract Monitoring with Rule-based Sentinels. https://docs.openzeppelin.com/defender/module/monitor. Accessed: 2026-04-06.

[8] Peckshield. 2021. bZx Hack Full Disclosure. https://peckshield.medium.com/bzx-hack-full-disclosure-with-detailed-profit-analysis-e6b1fa9b18fc. Accessed: 2026-04-30.

[9] Kaihua Qin, Liyi Zhou, Benjamin Livshits, and Arthur Gervais. 2021. Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit. In *International Conference on Financial Cryptography and Data Security*. Springer, Springer, Grenada, 3–32.

[10] Bin Wang, Han Liu, Chao Liu, Zhiqiang Yang, Qian Ren, Huixuan Zheng, and Hong Lei. 2021. BLOCKEYE: Hunting for DeFi Attacks on Blockchain. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, IEEE, Virtual, 17–20.

[11] Maoyi Xie, Ming Hu, Ziqiao Kong, Cen Zhang, Yebo Feng, Haijun Wang, Yue Xue, Hao Zhang, Ye Liu, and Yang Liu. 2024. DeFort: Automatic Detection and Analysis of Price Manipulation Attacks in DeFi Applications. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Vienna, Austria, 1517–1529.

[12] Juantao Zhong, Daoyuan Wu, Ye Liu, Maoyi Xie, Yang Liu, Yi Li, and Ning Liu. 2025. DeFiScope: Detecting Various DeFi Price Manipulations with LLM Reasoning. *IEEE Transactions on Dependable and Secure Computing* (2025). Available at https://ieeexplore.ieee.org/document/11334472; preprint `arXiv:2502.11521`.

[13] Liyi Zhou, Kaihua Qin, Christof Ferreira Torres, Duc Viet Le, and Arthur Gervais. 2021. High-Frequency Trading on Decentralized On-Chain Exchanges. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 428–445.

[14] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. 2023. SoK: Decentralized Finance (DeFi) Attacks. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, 2444–2461.